

TYPES & VARIABLES!

(LECTURE)



REVIEW: VARIABLES

```
name = "Joel"
```

It creates a **variable** named `name` holding the value `"Joel"`.

variable $\leftrightarrow$ box with a name attached to it.

name

Joel

PRINTING VARIABLES

We can print variables, similar to how we print strings or numbers:

```
course_number = 1100  
print(course_number) # 1100
```



```
1100
```

`print()` displays the *value* stored inside of the variable, not its *name!*

REVIEW: VARIABLES

```
name = "Joel"
```

The value within the box can change over the lifetime of the program (i.e. while it is being run or executed).

- variable only stores one value at a time
- previous values are forgotten forever

name

Joel

VARIABLES CHANGE

Variables can change over the course of a program, and can only hold one value.

Consider:

```
name = "Harry" # line a <-----  
name = "Joel"  # line b  
print(name)
```

name

Harry

VARIABLES CHANGE

Variables can change over the lifetime of a program, and can only hold one value.

Consider:

```
name = "Harry" # line a
name = "Joel"   # line b  <----
print(name)
```

name

Joel

LECTURE ACTIVITY

S7 How many different values will `fruit` have stored by the time we call the `print()` function? What value is printed?

```
fruit = "apple"
fruit = "banana"
veggie = "broccoli"
veggie = "lettuce"
fruit = "pear"
fruit = "tomato"
print(fruit)      # printing here!
fruit = "Pitaya"
```

Remember: code runs sequentially from top to bottom, one line at a time.

LECTURE ACTIVITY: VARIABLE NAMING AND STYLE

We follow `lower_snake_case` when naming variables: lowercase letters, words separated with underscores, and digits only after the first character.

M1:

Consider the following variables. Mark all names that are both legal (correct python syntax) and follow `lower_snake_case`?

- a) `local_counter3`
- b) `exec_and_fork`
- c) `AwesomeVariable`
- d) `awesome_Variable`
- e) `import`

QUICK ASIDE: WHY CAN'T WE USE IMPORT?

Some special words that are reserved for special purposes in Python!

These are called **keywords**.

As mentioned before, we use an `import` keyword to tell the computer we would like to use PennDraw in our Python program.

```
import penndraw as pd # 'import' and 'as' are two key words here
pd.line(1, 1, 0, 0)
pd.point(.5, .75)
pd.run()
```

Here are the reserved keywords, but *don't worry about memorizing.*

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

REVIEW: EXPRESSIONS

- **Expressions** are portions of a program that evaluate to ("have") a value.
- Basic expressions are composed of **literals**, **variables**, and **operators**

TERM	DEFINITION	EXAMPLE
Literal	A part of an expression that has a value which can be interpreted <i>literally</i>	4.0 or "python"
Variable	A named portion of memory that stores some value	year, x, or last_name
Operator	A symbol defining an operation or transformation	=, +, *, or <

REVIEW: OPERATORS & LITERALS

Yell it out: literal, variable, or operator?

- `"hello_there"`
- `=`
- `"+"`
- `2`
- `password_strength`
- `/`
- `"3"`
- `room_size`

REVIEW: F-STRINGS

f-strings are string literals that have an `f` prefix.

This allows us to have `{}` inside the string that contains an expression. That expression will be evaluated into its string representation.

```
y = 13
items = "bagels"
print(f"A baker's dozen is {y} things.")
print(f"A real dozen is {y - 1} things!")
print(f"I want to buy {y} {items}.")
```

```
A baker's dozen is 13 things.
A real dozen is 12 things!
I want to buy 13 bagels.
```

LECTURE ACTIVITY

L11 What is printed?

```
x = 101
print(f"{x - 100}")
print(f"(x + 1) is equal to ({x} + {1})")
```

TYPES

The **type** of a variable determines how the value it has stored is interpreted and used.

DATA TYPE	PURPOSE	SAMPLE VALUES	SAMPLE OPERATIONS
int	whole (integer) numbers	3, -14, 0	+, -, *, /
float	numbers with fractional parts	3.0, -14.32, 0.0	+, -, *, /
str	text	"CIS 1100", "False"	len(), indexing & slicing

STRINGS

We have seen strings before, they contain a sequence of characters.

Even in our first program, we were using a string:

```
print("Hello World!")  
my_string = "hey!"  
print(my_string)
```

USING OPERATORS ON STRING TYPES

When using the `+` operator on strings:

- the two strings are **concatenated**, or joined together from left to right.
- no spaces are inserted between them

```
example = "hello" + "world"  
print(example)
```



```
helloworld
```

OPERATOR + WITH STRINGS

`+` is used as an operator to append strings together

It does not *overwrite* or modify any of the variables that are its *operands*.

Example:

```
x = "cis1100"  
y = "com"  
z = x + "." + y  
print(x) # "cis1100"  
print(y) # "com"  
print(z) # "cis1100.com"
```

We only change what a variable stores using the `=` operator.

CALLING FUNCTIONS ON STRINGS

We can also call functions *on* strings to perform specific operations like so: `<string>.func_name()`

```
x = "abcdefghijklmnop"
y = x.upper()
z = "boYmeEtsWoRld".lower()
w = "hey! What's up?".upper()
print(x)    # "abcdefghijklmnop"
print(y)    # "ABCDEFGHIKLMNOP"
print(z)    # "boymeetsworld"
print(w)    # "HEY! WHAT'S UP?"
```

We can use these functions 'on' anything that is a string!

MORE STRING FUNCTIONS

- `.upper()` / `.lower()` makes a copy where all letters are upper/lowercase
- `.capitalize()` makes a copy with its first character capitalized and the rest lowercased.
- `.strip()` makes a copy where all space before and after the characters are removed.
 - e.g. `" hey! ".strip()` becomes `"hey!"`
- `str.replace(old, new)` makes a copy where all instances of the string `old` are replaced by `new` e.g. `"aaa".replace("a", "b")` evaluates to `"bbb"`
- `a + b` makes a new string value that has the value of `b` attached to the end of `a`

COMMON MISTAKES WITH VARIABLES

A couple of beginner mistakes to make when programming are:

- Forgetting that (most) operators do not modify variables that are operands
- improperly keeping track of values stored in variables

Let's take a look at a couple of problems!

LECTURE ACTIVITY

What is printed at the end of the program?

S8:

```
first = "    YOU are all    ".strip()
first = first.lower()
second = first.replace("you", "You")
second = second.replace(" ", " ")
third = second + " AMAZING!!!"
final = third.replace("!!!", "!")
print(final.capitalize())
```

You can delete a sequence from a string by replacing it with an empty string.

```
"12345".replace("3", "")
```



```
"1245"
```

C12 Write a few lines of code that:

1. asks the user to type a message
2. removes any instance of the sequence `"bad"`
3. prints out `"no bad words: "` followed by their message.

e.g. running `python no_bad_words.py`

```
$ Type something:
```

You can delete a sequence from a string by replacing it with an empty string.

```
"12345".replace("3", "")
```



```
"1245"
```

C12 Write a few lines of code that:

1. asks the user to type a message
2. removes any instance of the sequence `"bad"`
3. prints out `"no bad words: "` followed by their message.

e.g. running `python no_bad_words.py`

```
$ Type something: troubador
```

You can delete a sequence from a string by replacing it with an empty string.

```
"12345".replace("3", "") ➡ "1245"
```

C12 Write a few lines of code that:

1. asks the user to type a message
2. removes any instance of the sequence `"bad"`
3. prints out `"no bad words: "` followed by their message.

e.g. running `python no_bad_words.py`

```
$ Type something: troubador  
no bad words: trouor
```

NUMERICAL TYPES

In python, we can store numbers in variables. However, there is a distinction between two types:

- `int` These are Integers, meaning any positive or negative value (or zero).
 - e.g. `0`, `-3200`, `10`, `299792458`
- `float` These can store rational numbers and some special values
 - e.g. `3.14`, `8.3144`, `1.4142`, `2.718`, `infinity`, `-infinity`

NUMERICAL OPERATORS

- **+** addition
 - $x + y$
- **-** subtraction
 - $x - y$
- **/** divide
 - x / y
- ***** multiplication
 - $x * y$

Order of operations (PEMDAS) and evaluating from left to right still applies.

If you want to enforce what happens first or a specific order, use **(** and **)**.

MIXING NUMERICAL TYPES

- If you use an operator on two `int`s you get an `int`
 - (except `/` then you get a `float`, why?)
 - *the motivation for this might not be clear yet!*
- If you use an operator on two `float`s, the result will be a `float`
- if you operate on an `int` and a `float` you get a `float` (why?)

LECTURE ACTIVITY

What are the resulting types of the expressions and what will be printed?

S9:

```
x = 3 + 0.5 * 2  
print(x)
```

S10:

```
x = (2 * 8) / 3  
print(x)
```

MORE ASSIGNMENT OPERATORS

There are also a few more operators worth covering:

- `+=`

```
x = "h"  
x += "i"  
// x here becomes "hi"
```

This operator "adds" the two values together and sets the variable on the left equal to the result.

- Other variants: `-=`, `*=` and `/=` exist for numerical types (`*=` works on strings too!)

OTHER ARITHMETIC OPERATORS

- `**` used for exponents.
 - e.g. 5 squared is written as `5 ** 2`
- `//` used for "integer division, rounds the result towards 0"
 - `int // int` evaluates to an `int`
 - `3 // 2` evaluates to `1`
- `%` called "modulo" used to get the remainder of a division.
 - `5 % 2` evaluates to `1`
 - `9 % 3` evaluates to `0`

Lecture Activity

S11:

What does this evaluate to?

`(10 % 3) ** 2 // 5`

IF TIME: BOOLEAN TYPE

```
x = True  
y = False  
print(x)
```

COMPARISON

A common way to get boolean values is through comparison.

- `==` checks if two things are equal
- `!=` checks if two things are NOT equal

`"Hello" == "hello"` evaluates to `False`

`5 != 3` evaluates to `true`

`"hi" == "hi"` evaluates to `True`

More on `bool` & a new type `None` next time

REMINDER:

- Next lecture on Monday
- There is another check-in due before that lecture as well.
- Office Hours and Recitation start next week
 - Recitation attendance is counted, show up to your assigned recitation!
- HW00 is out and due Wednesday (9/10) at midnight